Autonomous CI/CD Incident Triager & Self-Healer

Problem Discussion Document

1. Introduction

Modern software systems use **CI/CD pipelines** to deploy frequent software updates efficiently. However, frequent deployments increase the risk of production incidents caused by code changes, configuration errors, or infrastructure failures. When an incident occurs, DevOps engineers must manually investigate logs, monitoring dashboards, deployment history, and code changes to identify the root cause. This process is time-consuming because the required information is spread across multiple tools and platforms. As systems grow in complexity, manual incident investigation leads to longer recovery times and increased downtime.

2. Problem Statement

Production incident diagnosis involves collecting and correlating data from logs, monitoring tools, deployment pipelines, and source code repositories. This manual process is slow, repetitive, and becomes increasingly difficult as system complexity grows. Delayed root cause identification increases downtime and operational costs. An automated incident triaging system can accelerate diagnosis and support faster recovery.

3. Background

A typical software deployment pipeline consists of code development, source code management, continuous integration, automated testing, build generation, deployment, and production monitoring. Failures can occur at any stage due to build errors, failed tests, deployment issues, configuration mistakes, infrastructure failures, or application defects. Once deployed, monitoring systems continuously track application health using metrics such as CPU usage, memory consumption, request latency, error rates, and service availability. When these metrics exceed predefined thresholds, monitoring tools generate alerts for the operations team. The responsibility then shifts to engineers to investigate the incident, identify the root cause, and restore normal system operation as quickly as possible.

4. Existing Incident Response Process

When a production alert is triggered, engineers must investigate the incident by collecting information from multiple sources. This typically involves:

- Reviewing monitoring dashboards and infrastructure metrics.
- Analyzing application and system logs.
- Checking recent deployments and CI/CD pipeline history.
- Examining recently merged code changes.
- Investigating configuration updates.
- Verifying the health of containers or virtual machines.
- Comparing the current deployment with the previous stable version.
- Consulting developers when additional context is required.
- Identifying the root cause of the incident.
- Determining the appropriate recovery action.

Since these activities are performed across multiple disconnected tools, the investigation process is largely manual and time-consuming. Engineers must correlate information from different sources to understand what caused the failure. This increases the time required to diagnose incidents, raises the risk of overlooking critical evidence, and delays service recovery. As software systems become more complex, manual incident triaging becomes increasingly inefficient and difficult to scale.

5. Challenges in Current Systems

Current monitoring and alerting tools can detect incidents but typically do not identify their root cause. Operational data is spread across multiple platforms, including monitoring dashboards, logs, deployment pipelines, source code repositories, and cloud infrastructure. Engineers must manually correlate information from these sources to diagnose issues. This investigation process is time-consuming and repetitive. The complexity of modern microservices-based systems further increases the difficulty of root cause analysis. Delayed diagnosis leads to longer service outages and higher operational costs. The investigation process also relies heavily on the experience and expertise of engineers. As a result, incident resolution remains slow despite advances in monitoring technologies.

6. Motivation

Organizations strive to minimize two key operational metrics: **Mean Time to Detect (MTTD)** and **Mean Time to Recover (MTTR)**. While modern monitoring tools can quickly detect failures, identifying their root cause remains largely a manual process. Engineers must analyze logs, metrics, deployment history, and code changes across multiple tools before taking corrective action. This investigation is time-consuming and directly increases recovery time. An intelligent incident response system can automate evidence collection, correlate information from different sources, and assist engineers in identifying the probable cause of an incident. This reduces manual effort, accelerates recovery, and improves overall system reliability.

7. Proposed Solution (High-Level)

The proposed project aims to develop an AI-driven incident triaging and self-healing assistant for DevOps environments. Upon receiving an operational alert, the system will automatically collect relevant data from monitoring tools, application logs, deployment history, source code repositories, and infrastructure events. It will correlate this information to identify the most probable root cause of the incident. The system will generate a concise incident summary, highlight supporting evidence, and recommend appropriate recovery actions. Suggested actions may include deployment rollback, service restart, or notifying the responsible development team. The objective is to reduce manual investigation effort, minimize Mean Time to Recovery (MTTR), and assist engineers in making faster, data-driven incident response decisions.

8. Expected Benefits

The proposed system aims to provide the following benefits:

- Faster incident investigation
- Reduced Mean Time to Recovery (MTTR)
- Automated evidence collection
- Improved operational efficiency
- Reduced manual workload for DevOps engineers
- Faster identification of probable root causes
- Better utilization of historical operational knowledge
- Improved system reliability and availability

9. Scope of the Project

The project focuses on assisting engineers during the incident investigation process rather than replacing human decision-making.

The proposed system will:

- Analyze operational information from multiple sources
- Correlate incident-related evidence
- Generate incident summaries
- Suggest probable root causes
- Recommend suitable recovery actions

The project will not initially focus on fully autonomous production deployments or unrestricted automatic infrastructure modifications. Human approval may still be required before executing corrective actions.

10. Conclusion

Modern cloud-native applications and CI/CD pipelines have increased the frequency and complexity of production incidents. Investigating these incidents requires engineers to manually analyze logs, metrics, deployment history, and code changes across multiple tools. This process is time-consuming, repetitive, and delays incident resolution. The proposed project aims to develop an AI-assisted incident triaging system that automatically collects and correlates operational data to identify probable root causes. The system will generate concise incident summaries and recommend appropriate recovery actions. By reducing manual investigation effort, the project seeks to decrease incident response time, improve operational efficiency, and enhance the reliability of modern software systems.